Relevant Section Identification — Instructions to Run

Live application: https://relevant-section-identification-qqgcuwkcmq-uc.a.run.app

Source code: https://github.com/ns-0437/relevant-section-identification

> *The deployed app is the fastest way to see it working — no setup at all. Note the **first query takes about 70 seconds**: the service scales to zero and downloads its embedding model on a cold start. Every query after that returns in well under a second. If it looks stuck on first load, it isn't.*

Everything below is for running it locally.

---

1. Prerequisites

Python 3.10 or 3.11. These are the versions with reliable wheels for `unstructured-inference` and `torch`.

Two system binaries must be on `PATH`:

Binary	Used for	Install
`tesseract`	OCR of figures	**Windows:** UB Mannheim installer · **macOS:** `brew install tesseract` · **Debian/Ubuntu:** `sudo apt install tesseract-ocr`
`pdftoppm` (poppler)	rasterising PDF pages	**Windows:** poppler-windows, add `Library\bin` to PATH · **macOS:** `brew install poppler` · **Debian/Ubuntu:** `sudo apt install poppler-utils`

Verify both:

```
tesseract --version
pdftoppm -v
```

On Windows the application locates Tesseract automatically if it is installed but missing from `PATH`, so a failure here is not fatal.

---

2. Get the code

```
git clone https://github.com/ns-0437/relevant-section-identification.git
cd relevant-section-identification
```

---

3. Create the environment

```
python -m venv .venv
```

Activate it:

Windows	`.venv\Scripts\activate`
macOS / Linux	`source .venv/bin/activate`

If you have an NVIDIA GPU, install torch first

The default PyPI wheel is CPU-only. Note that `pip install --upgrade torch --index-url .../cuXXX` **silently does nothing** if that index's newest torch is older than the CPU build you already have — the exact variant has to be pinned:

```
pip install --force-reinstall torch==2.13.0+cu126 torchvision --index-url
https://download.pytorch.org/whl/cu126
```

Choose the CUDA series your driver supports (`nvidia-smi` reports the maximum). Any CUDA 12.x wheel runs on a 12.x driver; CUDA 13 wheels need a 580+ driver.

A GPU is **optional**. It makes indexing roughly 7x faster and makes no measurable difference to query speed.

Install the dependencies

```
pip install -r requirements.txt
```

---

4. Authenticate for the embedding model

`google/embeddinggemma-300m` is a gated model. Accept the licence at https://huggingface.co/google/embeddinggemma-300m while signed in, then:

```
hf auth login
```

Paste a **read** token when prompted. The other two models — LLaVA for figure captions and Qwen for the chat tab — are ungated and download automatically on first use. **No API keys are required anywhere.**

---

5. Run the application

```
uvicorn app.main:app --port 8000
```

Open http://localhost:8000.

The sample 40-page datasheet is pre-indexed and loads by default, so you can type a question immediately. The first query takes around a minute while the embedding model loads into memory; after that it responds in a fraction of a second.

Try any of these:

- *What battery chemistries are supported by this charger?*
- *What resistor value sets the top-off current?*
- *What happens if the chip gets too hot?*
- *How can an MCU enable and disable battery charging?*

To use a different default document, set the environment variable `RSI_SAMPLE_PDF=/path/to/your.pdf` before starting the server.

---

6. Verify the installation (optional)

```
python -m pytest tests/ -q
```

15 model-free tests covering table rendering, split overlap, boilerplate detection and keyword extraction. They complete in about a second and need no models or GPU.

---

7. Using the interface

Panel	What it does
Viewer (left)	Renders the PDF. Page controls and zoom at the top.
Find sections (right)	The main feature. Returns relevant pages with their section headings and the text that matched. Clicking a result opens that page in the viewer.
Ask *(bonus)*	Answers the question in prose from the retrieved pages, with clickable page citations. Slow on CPU (30–60 s).
Upload PDF	Indexes a new document in the background with a progress indicator. Takes several minutes — the layout pass is CPU-bound and every figure is captioned.

---

8. Running the pipeline from the command line

The web app is not required. Each stage can be run on its own.

Chunk a PDF into text, table and figure chunks:

```
python pdf_chunker.py yourfile.pdf --device cuda --cache-dir .cache --out chunks.json
```

Embed those chunks and index them:

```
python embed_index.py chunks.json --db ./chroma --collection mydoc --device cuda --reset
```

Search from the terminal:

```
python hybrid_search.py "what does the STAT pin indicate?" --alpha 0.6
```

Reproduce the evaluation:

```
python -m evaluation.eval_v2 --collection max77751_v3 --by-kind
```

Reproduce the performance measurements:

```
python benchmark.py
```

Use `--device cpu` on any of these if you have no GPU. Drop `--device` entirely to let it choose automatically.

Useful flags on `pdf_chunker.py`:

Flag	Effect
`--cache-dir .cache`	Reuses the expensive layout pass on re-runs. Strongly recommended.
`--skip-images`	Skips figure captioning entirely — much faster.
`--keep-boilerplate`	Leaves running headers in, reproducing the earlier behaviour.
`--tables-from ocr`	Uses OCR for tables instead of the PDF text layer.
`--strip-base64`	Omits image data from the JSON, for readable output.

---

9. Expected timings

Measured on an RTX 3050 Laptop (4 GB VRAM) with 16 CPU cores.

Stage	GPU	CPU
Layout parse, 40 pages	~15 min	~15 min (CPU-bound either way)
Captioning 46 figures	~4 min	very slow — use `--skip-images`
Embedding 104 chunks	10 s	71 s
Search, warm	~0.12 s	~0.12 s

Indexing a fresh 40-page PDF end to end takes roughly 20 minutes the first time. With `--cache-dir` set, re-running afterwards takes about 4 minutes.

---

10. Troubleshooting

Symptom	Cause and fix
`TesseractNotFoundError`	Tesseract is not on `PATH`. See step 1.
`Unable to get page count. Is poppler installed?`	Poppler's `bin` directory is not on `PATH`.
`GatedRepoError` / 401 on EmbeddingGemma	The licence has not been accepted, or `hf auth login` has not been run. See step 4.
`torch.cuda.is_available()` returns `False` despite a GPU	A CPU-only torch wheel is installed. See the pinning note in step 3.
First query hangs for a minute	Expected — the embedding model is loading. Subsequent queries are fast.
Chat tab takes 30–60 seconds	Expected on CPU. A 1.5B model spends most of that on prompt processing.
`OSError` on import of `unstructured`	`pip install --force-reinstall unstructured-inference` usually resolves it.

---

11. What to look at first

If you have five minutes:

1. Open the live URL, wait out the first query, then ask *"What battery chemistries are supported by this charger?"* — it should return pages 17 and 19.
2. Click a result and watch the viewer jump to that page.
3. Switch to the **Ask** tab and run the same question.

If you have longer, `docs/REPORT.md` documents the method, the measurements, the three techniques that were tried and rejected, and the limitations.